

Structural Admissibility for Verification Systems: Observation, Warrant Debt, and Factorisation Repair

Abstract

Safety claims are usually made through an observation interface: a monitor, a log, a trace, a sensor, or an abstraction. Such interfaces do not preserve every concrete distinction. This matters when a safety predicate distinguishes states that the interface identifies. We call a predicate admissible relative to an observation map when it factors through that map. If factorisation fails, the missing distinction cannot be recovered by post-processing the observation. We formalise this criterion in Rocq. The development contains 41 Rocq source files, all compiling to corresponding `.vo` files, with no admitted lemmas and eight named axioms. The case studies are deliberately small. They cover consensus under partition, GPU-style relaxed memory observations, and physical sensor boundary models. They do not claim to be full models of those domains. Their purpose is narrower: to show how the same factorisation failure can appear in discrete, concurrent, and physical settings. The remedy is also ordinary: weaken the claim until the observation supports it, or strengthen the observation until the distinction is visible. The contribution is to state this discipline explicitly and to mechanise the reason that some apparent noise is a failed observation boundary. The development also makes the repair obligation operational: extracted monitor and certificate structures classify admissibility failures as engineering actions, while warrant debt is classified by kind, repair pressure, and expected repair burden.

1 Introduction

Verification rarely sees the system directly. It sees an abstraction, a trace, a log, a monitor state, a sensor reading, or a reconstructed state estimate. Each is an observation map. Each preserves some distinctions and collapses others. This is not an error. It is the normal condition under which verification and runtime assurance operate.

The risk begins when a safety predicate depends on a distinction that the observation does not preserve. Then the safety claim is stronger than the interface through which it is made. In routine engineering this problem

is often handled pragmatically. One restricts the operating envelope, adds a sensor, changes a monitor, rejects an over-strong claim, or records an extra field in a log. These are familiar moves. The paper gives them a simple structural test.

The same test can be used prospectively. Given a proposed claim and an observation interface, the factorisation criterion asks whether the claim is supported by what the interface actually carries. If not, the development classifies the repair obligation: restrict the claim, reject it, or refine the observation boundary.

A safety predicate is admissible relative to an observation map exactly when it factors through that map. Equivalently, states that have the same observation must agree on the predicate. If two concrete states look the same to the observation but differ with respect to the safety predicate, the claim is not supported by that observation. No downstream classifier, dashboard, optimiser, or monitor can repair the loss. The information has already been collapsed.

This point is modest, but important. Many admissibility failures will look like noise at the observational level. A sensor appears imprecise. A trace appears partial. A local execution appears harmless. A monitor appears inconclusive. The paper argues that this is sometimes the wrong diagnosis. The problem may not be noise. It may be a failed factorisation.

The paper does not propose a new general-purpose verification algorithm. It does not claim that all safety reasoning reduces to factorisation. It isolates a necessary condition on safety claims made through observation interfaces. When that condition fails, the claim must be restricted, rejected, or the observation must be refined.

Contributions.

- We define structural admissibility as factorisation of a safety predicate through an observation map.
- We prove the non-recoverability principle: once an observation collapses a safety-relevant distinction, post-processing cannot separate it.
- We mechanise the framework in Rocq, with 41 compiled source files, no admitted lemmas, and eight named axioms.
- We give small case studies in consensus, GPU-style memory observations, and physical sensor models.
- We distinguish two repair forms: predicate rejection or restriction, and observational refinement. These are formally parallel but not operationally symmetric.

- We expose this repair discipline through extracted engineering-action records, so that monitor and certificate outputs can be read as explicit repair obligations rather than as unstructured warnings.
- We repackage the physical threshold assumptions as instances of a general non-degenerate-threshold obligation, making clear that the issue is mathematical parameterisation, not an ad hoc feature of the physics example.
- We refine warrant debt from a mnemonic into a qualitative typology, classifying debt by kind, repair pressure, and repair cost band.

Section 2 states the factorisation criterion. Section 3 defines warrant debt and non-recoverability. Section 4 explains restriction and refinement. Section 5 describes the Rocq development. Section 5.5 reports the case studies. Sections 7 through 10 place the result, state its limits, and draw the practical lesson.

Theorems are proved about the stated models. Industrial relevance follows only when an engineering setting instantiates the same observation, predicate, collapse, and repair structure.

2 Mathematical Setting

Let X be a concrete state space, O an observation space, and $M : X \rightarrow O$ an observation map. Let $\Phi : X \rightarrow \text{Prop}$ be a safety predicate.

2.1 Observation-induced equivalence

The observation map M induces an equivalence on concrete states:

$$x \sim_M y \quad \text{iff} \quad M(x) = M(y).$$

States that agree under M are indistinguishable from the perspective of the observation interface.

2.2 Admissibility

Definition 2.1 (Admissibility). *The predicate Φ is admissible relative to M if and only if*

$$M(x) = M(y) \implies (\Phi(x) \leftrightarrow \Phi(y)).$$

Equivalently, Φ is constant on the fibres of M . This is a support condition: it says that the observation contains enough information for the predicate being asserted through it.

2.3 Factorisation

Definition 2.2 (Factorisation). *The predicate Φ factors through M if there exists $\widehat{\Phi} : O \rightarrow \text{Prop}$ such that*

$$\forall x : X, \quad \Phi(x) \leftrightarrow \widehat{\Phi}(M(x)).$$

2.4 The main equivalence

Theorem 2.3 (Admissibility iff factorisation). *The predicate Φ is admissible relative to M if and only if it factors through M .*

Proof. If Φ factors through M , then $M(x) = M(y)$ immediately gives $\Phi(x) \leftrightarrow \Phi(y)$. Conversely, suppose Φ is constant on the fibres of M . Define

$$\widehat{\Phi}(o) :\Longleftrightarrow \exists x : X. M(x) = o \wedge \Phi(x).$$

Then $\Phi(x)$ implies $\widehat{\Phi}(M(x))$ by taking x as witness. Conversely, if $\widehat{\Phi}(M(x))$ then for some y , $M(y) = M(x)$ and $\Phi(y)$. Fibre-constancy gives $\Phi(x)$. In the Rocq development this is `admissible_iff_constant_on_kernel`. $\square$

This equivalence is the formal spine of the paper. The rest of the paper is an application of it. Each case study asks whether a predicate is constant on the fibres of the stated observation.

3 Warrant Debt and Non-Recoverability

3.1 Warrant debt

Definition 3.1 (Witnessed warrant debt). *An observation map M has witnessed warrant debt with respect to Φ when there exist $x, y : X$ such that*

$$M(x) = M(y), \quad \Phi(x), \quad \neg \Phi(y).$$

A witnessed warrant debt is a concrete certificate of non-admissibility. The term is meant literally. A safety claim made through M borrows distinctions that M does not provide.

Definition 3.2 (Profiled warrant debt). *A profiled warrant debt is a witnessed warrant debt together with a classification of the collapsed distinction and the corresponding repair pressure. The classification does not measure risk numerically. It records what kind of information has been lost and what sort of engineering act would be required to discharge the debt.*

The point of the profile is not to price safety numerically. It prevents a category error. A thread-local trace debt and a boundary-sensing debt may instantiate the same formal pattern, but they do not impose the same repair cost. The first may require an extra log field or a restricted execution class. The second may require additional instrumentation or a stronger physical model.

Debt kind	Collapsed distinction	Typical repair	Repair burden
Quorum debt	Global quorum structure	Enrich quorum data or restrict to local belief	Moderate
Trace debt	Global ordering or visibility	Add ordering metadata or restrict to DRF executions	Low to moderate
Boundary debt	Boundary or inter-sample behaviour	Add sensing, margins, or boundary data	High
Threshold debt	Non-degenerate threshold condition	State threshold obligation explicitly	Low, proof-critical
Runtime debt	Fatigue or rupture-state information	Emit review or rupture action	Operational

Table 1: Qualitative warrant-debt profiles. The table is not a quantitative risk metric. It classifies the kind of collapsed distinction and the corresponding repair pressure.

3.2 Non-recoverability

Theorem 3.3 (Non-recoverability). *For any map $f : O \rightarrow Y$,*

$$M(x) = M(y) \implies f(M(x)) = f(M(y)).$$

Consequently, if M collapses x and y , no downstream map from observations can separate them.

Proof. By congruence of equality under f . In the Rocq development this is the non-recoverability lemma for post-processing maps over observations. $\square$

Corollary 3.4 (No post-processing repair). *Suppose $M(x) = M(y)$, $\Phi(x)$, and $\neg\Phi(y)$. Then for every $f : O \rightarrow Y$, the observation $f \circ M$ does not make Φ admissible.*

Proof. Since $M(x) = M(y)$, congruence gives $f(M(x)) = f(M(y))$. But Φ separates x and y . Hence Φ is not constant on the fibres of $f \circ M$. $\square$

No function of the observation can recover a distinction not present in the observation. Classifiers, dashboards, monitors, and optimisers are such functions unless they receive additional information.

3.3 Why this can be mistaken for noise

At the observational level, an inadmissibility witness may look unremarkable. Two executions have the same local trace. Two sampled trajectories have the same visible readings. Two distributed states have the same local view.

It is tempting to describe the disagreement as measurement noise, modelling imprecision, or a benign abstraction artefact. Sometimes that is correct. But when the predicate separates two states that the observation identifies, the problem is structural. The observation is not merely noisy with respect to the claim. It is insufficient for the claim.

This is the practical reason for naming warrant debt. The name does not add a new proof principle. It marks a common mistake: treating a failed observation boundary as an implementation nuisance.

3.4 Consequence for verification

If a proof of Φ requires a distinction invisible to M , the proof is not a proof through M . It is a proof through a stronger, implicit observation. The framework makes this precise. An inadmissible predicate cannot be made admissible by adding an assumption that bypasses the observation boundary. The assumption merely moves the missing observation into the proof.

4 Recovery: Restrict or Refine

At the level of the factorisation criterion, there are two principled repair forms.

4.1 Predicate rejection or restriction

One may replace Φ by a predicate Ψ that is admissible relative to the existing observation M . This may mean weakening the guarantee, restricting the state class, strengthening the operating envelope, using robust margins, or limiting executions to those the observation can support.

Rejection does not always mean weakening the predicate in a crude sense. It means refusing to assert a claim whose truth is not determined by the available observation. Restriction is the constructive version of that refusal.

4.2 Observational refinement

Alternatively, one may replace M by a richer observation $M' : X \rightarrow O'$ together with a projection $\pi : O' \rightarrow O$ such that

$$M = \pi \circ M'.$$

The refined observation M' preserves the old observation while exposing additional structure. If Φ factors through M' , admissibility has been recovered by changing the observation boundary rather than the predicate.

4.3 Formal duality is not operational symmetry

Restriction and refinement are formally parallel repair forms. They are not the same engineering act. Restriction keeps the observation map fixed and changes the claim. Refinement keeps the claim fixed and changes the observation. Both can restore factorisation. They pay different costs.

Restriction pays in guarantee strength. The resulting claim may be narrower, conditional, local, robust only within a margin, or valid only under a specified synchronisation discipline. Refinement pays in observational burden. The system must expose more structure, log more information, add a sensor, preserve a trace, or reconstruct a richer state.

This distinction is visible in the case studies. Both repairs are mechanised in all three domains. They are not the same engineering act in all three domains. The repair that is natural in one setting may be artificial in another.

4.4 Everyday safety procedures already use both repairs

The distinction is not exotic. Everyday safety procedures already use both moves. A restricted operating envelope is predicate restriction: the claim is no longer that all states are safe, but that states inside a stated envelope are safe. An added sensor is observational refinement: the claim is kept, but the interface is strengthened. A lockout condition restricts admissible operation. A new alarm channel refines observation. A conservative threshold restricts the claim to a margin. A higher resolution measurement refines the observation.

The paper's point is not that engineers do not know this. They do. The point is that verification texts and monitor designs can obscure the distinction. Once the distinction is obscured, an unsupported safety predicate can be smuggled in as a harmless assumption, a modelling convention, or noise. The factorisation criterion is a discipline against that mistake.

4.5 Repair choice as an engineering obligation

The factorisation failure does not decide the repair automatically. It creates an obligation. The engineer must decide whether the unsupported distinction should be removed from the claim or added to the observation.

A restricted claim may be the correct engineering response when the stronger claim is not worth the instrumentation required to support it. A refined observation is the correct response when the stronger claim is required and the missing distinction can be exposed by logging, sensing, reconstruction, or synchronisation discipline. Explicit carriage is not a proof repair. It is the act of recording that a known admissibility debt remains attached to a weaker safety case.

Repair	Claim served?	pre-	Observation changed?	Cost paid
Restriction	No		No	Reduced guarantee strength
Rejection	No		No	Claim abandoned
Refinement	Yes		Yes	Added observation burden
Explicit car-riage	No full discharge		No	Documented limitation

Table 2: Repair choices after factorisation failure. Restriction and refinement are formally parallel repairs, but they are not operationally symmetric.

4.6 No invisible observation

Principle 4.1 (No invisible observation). When a predicate fails to factor through the current observation map, a proof must not be repaired by an assumption that supplies the missing distinction without changing the stated observation boundary.

The proof obligation is discharged only by restricting the claim, rejecting the claim, or refining the observation. The case studies in Section 5.5 illustrate this discipline.

5 Rocq Development

5.1 Library structure

The mechanisation is organised as follows:

Foundation/ Relations, orders, lattices, quotients, topology, mereotopology, and metric resolution.

Admissibility/ Observation maps, the factorisation criterion, kernel pairs, refinement, collapse witnesses, warrant debt, profiled warrant debt, and horizon debt.

Runtime/ Monitor states, structural fatigue, metastable closure, rupture certificates, recovery, and runtime soundness.

Automation/ Custom tactics for admissibility, refinement, collapse, and certificate reasoning.

CaseStudies/Consensus/ Network model, partition collapse, consensus admissibility.

CaseStudies/GPU/ Memory model, GPU admissibility.

CaseStudies/PhysicalSystems/ Boundary conditions, non-degenerate threshold obligations, thermal model, sensor collapse, PDE admissibility.

Extraction/ Extractable types, monitor extraction, certificate extraction, engineering-action extraction, and extraction correctness.

5.2 Formal status

The accompanying Rocq development consists of 41 source files, all of which compile to .vo files. It contains no admitted lemmas. The remaining logical strength of the development is confined to eight named axioms. They are not hidden. They are part of the mathematical specification.

5.3 Axioms and their role

The eight axioms are background principles, not incomplete proofs. Three concern primitive connection in mereotopology:

C.refl Every region is connected to itself.

C.sym Connection is symmetric.

ntpp_irreflexive No region is a non-tangential proper part of itself.

The axiom **ntpp_irreflexive** is an RCC-style mereotopological assumption. It is not derivable from **C.refl** and **C.sym** alone. It is therefore stated explicitly. The derived result **ntpp_antisym** is proved from it.

The remaining five axioms are application-level constraints. Their exact Rocq names and roles are given in Table 3. The following paragraphs explain how each axiom interacts with the actual proof obligations it discharges.

boundary_obs_respects_signature. This axiom is the contractual interface between the abstract sensor model and the factorisation failure proof. It states that regions sharing the same boundary signature produce identical boundary observations. The key use is in **boundary_collapse_induces_admissibility_failure**: the proof must rewrite the boundary observation of one region in terms of the other after establishing that their signatures agree. Without the axiom, that rewrite is not licensed. The axiom is not a consequence of the mereotopological base. It is an explicit requirement imposed on any concrete instantiation of the boundary sensor interface.

This is the physical analogue of a hidden structural bridge in the discrete case studies. In the GPU case, the restriction proof relies on the DRF-SC bridge for the toy execution model. In the physical case, the corresponding burden is boundary coherence and non-degenerate thresholds. The burdens differ in content, but they play the same methodological role: they state

Rocq axiom name	Role in the model	Why it is explicit
<code>C_refl</code>	Every region connects to itself	Base mereotopology
<code>C_sym</code>	Connection is symmetric	Base mereotopology
<code>ntpp_irreflexive</code>	No region is an NTPP of itself	Boundary discipline
<code>boundary_obs_respects_signature</code>	Same boundary signature implies same observation	Sensor interface coherence
<code>max_temp_positive</code>	Thermal safety threshold is positive	Non-degenerate witness
<code>max_pressure_positive</code>	Pressure safety threshold is positive	Non-degenerate witness
<code>max_flux_positive</code>	Flux safety threshold is positive	Non-degenerate witness
<code>meta_review_trigger_lt_fatigue_threshold</code>	Review trigger fires before hard rupture threshold	Runtime monitor ordering

Table 3: All eight named axioms with their exact Rocq names and roles. Full justification for each appears in `ASSUMPTIONS.md`.

the conditions under which the observation map is precise enough for the factorisation question to be well posed.

`max_temp_positive`, `max_pressure_positive`, `max_flux_positive`. These three axioms are instances of one structural obligation: a threshold used to build a predecessor witness must be non-degenerate. The proofs compare a threshold value with its predecessor. In Rocq’s natural-number arithmetic this comparison is meaningful only when the threshold is strictly positive. Otherwise subtraction by one saturates at zero, and the intended witness degenerates.

The development therefore packages the issue as a non-degenerate-threshold obligation. Temperature, pressure, and flux are three instances of the same mathematical requirement. The point is not that the physical model needs three unrelated physical assumptions. The point is that any threshold-based safety witness of this form must state the arithmetic condition under which the witness is non-degenerate.

`meta_review_trigger_lt_fatigue_threshold`. This axiom orders the two key parameters of the runtime monitor: the fatigue level at which the monitor enters `META_REVIEW`, and the level at which it enters `HARD_RUPTURE`. The strict inequality ensures that there exists a fatigue band in which the monitor is under structural review but has not yet triggered hard rupture.

Without this ordering, the `META_REVIEW` state could be unreachable: if the hard-rupture threshold were not above the review trigger, the monitor could cross directly from `CLOSED` to `HARD_RUPTURE`, bypassing the intermediate phase. The axiom governs the topology of the monitor state machine: it is the condition that makes metastable closure distinct from hard rupture.

The continuous case does more mathematical work than the discrete examples: boundary assumptions and ordering constraints have no direct analogue in a thread-local trace or a partitioned local view. The common claim is that each domain contains an observation map, a predicate, and a possible failure of factorisation.

5.4 Mechanisation scope

The development mechanises the factorisation criterion and its equivalence with admissibility, the non-recoverability principle, the runtime monitor theory including rupture certificates, and the three case-study examples. It does not claim to be a full mechanisation of Paxos, a production GPU memory model, or a continuous control-system specification.

Table 4 maps each prose claim to its Rocq artefact.

Claim	Rocq artefact	Status
Admissibility iff fibre-constancy	<code>admissible_iff_constant_on_kernel</code>	proved
Non-recoverability	non-recoverability lemma in <code>Collapse.v</code>	proved
Consensus refinement	<code>ConsensusAdmissibility.v</code>	proved
Consensus restriction	<code>ConsensusRestriction.v</code>	proved
GPU refinement	<code>GPUAdmissibility.v</code>	proved
GPU DRF restriction	<code>GPURestriction.v</code>	proved, conditional on DRF-SC bridge
Physical restriction	<code>PDEAdmissibility.v</code>	proved
Physical refinement	<code>PDEAdmissibility.v</code>	proved
Extraction bridge	<code>ExtractionCorrectness.v</code>	proved
Profiled warrant debt	<code>WarrantDebt.v</code>	proved/classified
Non-degenerate thresholds	<code>BoundaryConditions.v</code> , <code>ThermalModel.v</code> , <code>PDEAdmissibility.v</code>	parameterised
Engineering-action extraction	<code>MonitorExtraction.v</code> , <code>CertificateExtraction.v</code>	extracted

Table 4: Prose claims and their Rocq artefacts.

The profiling and action records are not additional safety theorems about the external world. They are typed interfaces around the existing

factorisation result. Their role is to prevent an admissibility failure from being reported as an unstructured warning. The reported object records the kind of debt, the repair pressure, and the corresponding engineering action.

5.5 Extraction bridge

The `Extraction/` component produces two OCaml files from the Rocq development via the built-in `Extraction Language OCaml` command: `monitor_extracted.ml` and `cert_extracted.ml`.

`monitor_extracted.ml` contains the function `update_monitor`, which takes an `ExtractableSnapshot` and an `ExtractableInput` and returns the next snapshot. The Rocq type `ExtractableSnapshot` records fatigue level, tension, horizon, lag, and numeric status code (0 = `CLOSED`, 1 = `META_REVIEW`, 2 = `HARD_RUPTURE`). The extracted file also contains the proofs of two verified properties: `update_monitor_preserves_invariants`, which states that the monitor invariant is maintained across every transition, and `update_monitor_hard_rupture_absorbing`, which states that once the hard-rupture status is set it is never cleared. The proof terms carry no operational content in the extracted program; the computational content is the monitor update function itself. The extracted code is therefore not a proof checker. It is code produced from terms whose relevant properties have been proved in Rocq.

`cert_extracted.ml` contains `extracted_cert_valid`, a boolean certificate checker, and `build_extractable_cert`, a constructor for rupture certificates from explicit witnesses. Within the Rocq model, the soundness result `extracted_certificate_sound` states that every certificate accepted by the checker satisfies the certificate predicate: the two predicate values recorded in the certificate differ, and the fatigue component is positive. It does not certify the external sensor pipeline, the OCaml compiler, or the runtime environment.

The correctness connection between the Rocq specification and the extracted code rests on three proved theorems in `ExtractionCorrectness.v`. `snapshot_round_trip` proves that converting a `MonitorSnapshot` to the extractable representation and back is an identity. The theorem `coq_extractable_coq_equiv` proves that a round-trip through the extractable types preserves both fatigue level and status code. Together they ensure that the type bridge is faithful: no information is discarded or corrupted by the representation change.

The extraction layer also exposes an engineering-action interface. A monitor status is mapped to an action classification: closed states require no repair action, meta-review states require review of the observation boundary, and hard-rupture states require rejection or refinement. Certificate output is likewise mapped to a repair obligation when it records a concrete predicate disagreement over an identical observation. This is not a dashboard imple-

Extracted condition	Engineering action	Interpretation
Closed	No repair action	Current observation supports the monitored state
Meta Review	Review observation boundary	Structural fatigue is visible before rupture
Hard Rupture	Reject, restrict, or refine	The current claim is no longer supportable
Certificate witness	Open admissibility obligation	Same observation, different predicate value

Table 5: Operational reading of extracted monitor and certificate output.

mentation. It is the typed core that a dashboard, issue tracker, or safety-case tool would consume.

The limitation of the extraction layer is the standard one. Rocq’s extraction theorem guarantees that the extracted OCaml has the same computational behaviour as the Rocq term under the Rocq reduction rules. It does not verify the OCaml compiler, the runtime system, or the interface code that feeds sensor data into `ExtractableInput` records. That boundary is the point where the formal guarantee ends and engineering practice resumes.

6 Case Studies: Refine or Reject

The development contains three case-study families: consensus under network partition, GPU-style relaxed memory, and cyber-physical sensor models. These examples are deliberately drawn from different engineering domains. Their common structure is the point. In each case, a safety predicate is naturally formulated over a concrete state space X , while the available observation map $M : X \rightarrow O$ exposes only a coarser interface. The case study then asks whether the predicate factors through that observation.

The answer is negative for the initial coarse observation in each family. The coarse observation identifies concrete states that disagree on the relevant safety predicate. The admissibility failure is therefore structural rather than accidental. Recovery takes one of two forms: replace the predicate by one that is meaningful at the observational grain, or refine the observation so that the safety-relevant distinction is no longer collapsed.

Mechanisation status. The three case-study families instantiate the same factorisation pattern. The consensus case study mechanises both repair paths: the refinement path restores admissibility for `quorum_safe` by enriching the observation (`ConsensusAdmissibility.v`), and the restriction path yields an admissible local-quorum-belief predicate at the coarse

Table 6: Case-study structure: factorisation failure and admissibility recovery.

Case study	Hidden distinction	Coarse failure	Recovery
Consensus	Partition and quorum structure	Local observations do not determine global quorum safety	Restrict to local quorum belief; or enrich with quorum-safety data
GPU memory	Buffer, cache, and ordering structure	Value traces do not determine consistency behaviour	Restrict to DRF executions; or enrich with global ordering metadata
Physical systems	Inter-sample and boundary behaviour	Sampled values do not determine continuous safety	Use robust observational margins; or enrich sensing and boundary data

observational grain (`ConsensusRestriction.v`). The GPU case study mechanises both repair paths: refinement by enriched global execution observation (`GPUAdmissibility.v`), and restriction to the subtype of data-race-free executions (`GPURestriction.v`). The physical-systems case study mechanises both paths: a restricted predicate is admissible at the coarse sensor grain, and an enriched sensor observation restores admissibility for the stronger predicate. The development mechanises predicate restriction in all three domains and observational refinement in all three.

6.1 Consensus Under Partition

The consensus case study concerns quorum safety under network partition. The concrete state records enough information to determine whether the relevant quorum-safety predicate holds. A coarse observation, however, may expose only local status information. Such an observation does not determine global quorum safety. Two concrete states may agree on the local observation while disagreeing on whether a quorum is safely established.

This failure is not a defect in the proof script. It is the phenomenon the framework is intended to detect. A uniformity principle for reachable alive nodes fails in the partition model: different nodes may have different reachable alive counts. The admissibility proof cannot be closed by that route.

The mechanised consensus case study proves both repair paths.

Refinement. The observation record `ConsensusObs` contains the field `co_quorum_safe`, computed by `consensus_obs` from the underlying consensus state. The proof of admissibility is then immediate: equality of observations entails equality of the quorum-safety component by projection. The safety predicate becomes admissible because the observation carries the

safety-relevant distinction directly.

Restriction. Rather than asserting global quorum safety, one can instead assert only what the local observation supports: the node’s own quorum belief, `local_quorum_belief n`, a projection of `lv_quorum` from the local view record. This predicate is admissible relative to `local_node_obs n` because it is a projection of the observation. The proof requires no axiom. The theorem `restriction_is_genuinely_weaker` confirms that local belief and global quorum safety can disagree, so this is a genuine weakening and not a reformulation.

The theorem `both_repairs_are_available` states the formal duality in a single proposition: both `local_quorum_belief n` and `quorum_safe` are admissible, the former relative to the coarse local observation, the latter relative to the enriched consensus observation.

6.2 GPU-Style Memory Observations

The GPU-style case study concerns the gap between value observations and memory consistency. A coarse observation may record only the values read and written by threads. The concrete execution state, however, contains additional ordering and visibility structure: buffering, cache state, scope, and synchronisation metadata. A predicate such as sequential consistency of the concurrent execution is not, in general, determined by the value trace alone.

The admissibility failure has the same form as in the consensus case. Two executions may have the same observed value trace for a given thread while disagreeing on whether the execution is sequentially consistent. The observation map has collapsed a distinction on which the safety predicate depends. The predicate therefore fails to factor through the thread-local observation.

The GPU case study mechanises both repair paths. Refinement restores admissibility by enriching the observation with global execution structure. Restriction keeps the thread-local observation fixed but restricts the domain to data-race-free executions. On that restricted domain, the sequential-consistency predicate is stable across observation fibres, conditional on the DRF-SC bridge stated in `GPURestriction.v`.

6.3 Physical Sensing and Boundary Conditions

The physical-systems case study concerns safety predicates over continuous or boundary-sensitive states. A controller or monitor rarely observes the full physical trajectory. It receives sampled, discretised, or otherwise compressed sensor data. A predicate may require that a variable remain within a safe region throughout an interval, while the observation records only values at selected sampling points or coarse boundary summaries.

The resulting admissibility failure is again a failure of factorisation. Two physical trajectories may induce the same sampled observation while disagreeing on the safety predicate. One trajectory may remain inside the safe envelope, while another may violate the envelope between samples or across an unresolved boundary. The sampled observation is therefore insufficient to determine the predicate.

This case study mechanises both recovery paths. The coarse sensor observation is proved inadmissible for the original safety predicate. A restricted predicate meaningful at the coarse observational grain is then shown to be admissible relative to that observation. Finally, an enriched sensor observation that adds the missing physical data restores admissibility for the stronger predicate. The physical-systems case study therefore demonstrates the full design choice: one may weaken the guarantee to match the observation, or strengthen the observation to match the guarantee.

6.4 Common Structure

The three case-study families differ in engineering content but not in logical shape. Consensus hides partition and quorum structure. GPU-style memory hides ordering and visibility structure. Physical sensing hides inter-sample or boundary behaviour. In each case, the initial observation map is too coarse for the safety predicate. The predicate fails to factor because the observation identifies concrete states that the predicate separates.

This is the common admissibility defect. It is independent of whether the hidden structure is distributed, concurrent, or physical. The repair is also common: either choose a predicate that is meaningful at the given observational grain, or refine the observation until the predicate becomes invariant on observation fibres. The case studies therefore support the main methodological claim of the paper. Admissibility is not merely a property of a predicate or a system. It is a property of the relation between a predicate and an observation interface.

6.5 Comparing the burden across domains

The case studies share a formal pattern. They do not share a mathematical substrate. In the consensus example, the relevant collapse is induced by local views under partition. In the GPU example, the relevant collapse is induced by thread-local observation of an execution whose global consistency status may differ. In the physical-systems example, the collapse is mediated by sampling, boundary behaviour, and mereotopological assumptions about regions.

The physical case is therefore the heaviest case, not because the criterion changes, but because the observation boundary is richer. Spatial connection, proper containment, inter-sample behaviour, and robust margins must be

stated before the factorisation question is even well posed. This is why the physical case has explicit RCC-style assumptions and application-level coherence constraints. The discrete cases need less geometric structure: they are simpler tests of the same criterion.

The case studies exercise both repair forms in all three domains. This symmetry is a mechanisation fact, not an engineering equivalence. In consensus, restriction becomes a local-quorum-belief predicate. In GPU-style memory, restriction becomes a data-race-free execution subdomain. In the physical case, restriction becomes a robust observational margin. The formal pattern is common. The cost of the repair is domain-specific.

This also explains the different burden of the restriction proofs. In the consensus case, restriction changes the claim from global quorum safety to local quorum belief. In the GPU case, restriction changes the domain from arbitrary executions to data-race-free executions, with the DRF-SC bridge stated as a proof obligation for the toy model. In the physical case, restriction is naturally expressed by margins and boundary conditions. The same repair form is present in all three cases. It is not the same engineering act in all three cases.

The non-degenerate-threshold parameterisation clarifies this asymmetry. The physical case does not require special pleading. It requires explicit parameterisation of the arithmetic and boundary conditions that make its observation space meaningful. The GPU case carries an analogous burden through the DRF-SC bridge. The consensus case carries it through the relation between local belief and global quorum structure. The common criterion is unchanged. What changes is the cost of making each observation boundary precise.

6.6 Structure of the observation-induced equivalences

The three case studies induce observation-equivalences with different mathematical structures, despite sharing the same logical form $x \sim_M y \Leftrightarrow M(x) = M(y)$. Making this contrast explicit clarifies why the physical case requires heavier axiomatisation and why restriction is more natural there.

Consensus. The equivalence $ns_1 \sim_{M_n} ns_2$, where $M_n = \text{local_node_obs } n$, holds when both network states produce the same `LocalView` record for node n . That record contains only a message list filtered by the partition relation and a local quorum boolean. Two states fall in the same fibre when node n cannot observe the global reachability information that separates them. The equivalence is combinatorial: it depends entirely on which messages survive a single Boolean filter.

GPU memory. The equivalence $e_1 \sim_{T_t} e_2$, where $T_t = \text{thread_local_obs } t$, holds when both executions produce the same sequence of memory events for thread t . The equivalence is sequential: it collapses all inter-thread ordering,

buffering, and synchronisation metadata. Two executions fall in the same fibre when the thread-local value trace is identical but the global consistency status differs. No axioms beyond those for list equality are required to state this equivalence.

Physical sensing. The equivalence $s_1 \sim_\sigma s_2$, where $\sigma = \text{sensor_obs}$, holds when both physical states share the same boundary pressure and flux readings, regardless of temperature. This is the simplest fibre condition to write down, but it is not the cheapest to reason about. Constructing the inadmissibility witness requires that $\text{max_temp} - 1 < \text{max_temp}$, which is only valid under the positivity axiom. Stating what a safe region *is*, before asking whether it is observable, requires the boundary and containment structure provided by the RCC-style parameters and axioms. The equivalence is geometrically grounded in a way the discrete equivalences are not.

The contrast is this. In the discrete cases the observation-induced equivalence is structurally transparent: list filtering and record projection. In the physical case the equivalence requires background geometric conditions to be non-degenerate. This is not a weakness of the physical model. It reflects the genuine cost of stating a spatial safety predicate precisely enough to ask the factorisation question at all.

7 Related Work

7.1 Abstraction and abstract interpretation

The nearest technical setting is abstract interpretation. Cousot and Cousot define program analysis by interpreting concrete computations in an abstract domain, with soundness controlled by the relation between concrete and abstract semantics [5]. The present paper uses only the following fragment of that setting: a predicate must be constant on the observational fibres if it is to be asserted through the observation.

Counterexample-guided abstraction refinement gives a standard method for repairing abstractions that admit spurious counterexamples [3, 4]. The criterion here is prior to refinement search: before asking how to refine, one asks whether the asserted predicate is even invariant on the current observational fibres. CEGAR asks how to refine after an abstract counterexample. Structural admissibility asks whether the predicate was observable through the interface in the first place.

7.2 Runtime verification and monitorability

Runtime verification studies what can be judged from finite or partial runtime observations. Bauer, Leucker, and Schallhart give a three-valued account for LTL and TLTL monitoring, including inconclusive outcomes over partial

traces [2]. This paper does not replace monitorability theory. It states a basic obstruction that any such theory must respect: a monitor cannot distinguish states that its observation interface identifies.

Functional safety and risk-management practice already distinguish between changing the operating claim and changing the instrumentation. Standards such as ISO 31000 and IEC 61508 are not factorisation theories, but they make clear that risk treatment, safety functions, monitoring, and operating conditions must be stated together [7, 6]. This paper supplies a small formal criterion for one version of that practical discipline.

7.3 Consensus and weak memory

The consensus case study is intentionally small. It should be read against the larger literature on distributed agreement, including Lamport’s presentation of Paxos [8]. The point is not to mechanise Paxos. The point is to show how a local observation can collapse states that differ with respect to a global safety predicate.

The GPU-style case is similarly modest. It is motivated by the general problem that weak-memory behaviours depend on global consistency constraints not visible in a local thread trace. Work on x86-TSO and weak memory models gives the broader context for this issue [9, 1]. Here again, the paper does not claim to mechanise a production memory model. It uses the setting to expose factorisation failure.

7.4 Type-theoretic and constructive verification

The mechanisation is carried out in Rocq, formerly Coq. Proof assistants make assumptions, definitions, and proof obligations explicit. That explicitness is part of the paper’s method. A hidden observational assumption is not harmless because it permits a theorem to compile. It changes the observation through which the theorem is being claimed [11].

7.5 Mereotopology and region-based spatial reasoning

The physical-systems case uses region-based reasoning in the spirit of the Region Connection Calculus. Randell, Cui, and Cohn’s RCC framework treats spatial reasoning in terms of regions and connection rather than point-set coordinates [10]. The present paper uses only a small part of this tradition. Its role is to make boundary-sensitive observation explicit.

8 Limitations

The case studies are small by design. Their role is separation, not coverage: each isolates a concrete observation map, a predicate, a collapse witness, and

a repair. No result depends on treating these models as complete industrial specifications.

Both repair forms are mechanised in all three domains. This is a coverage claim about the case studies, not a claim that the repairs have equal strength, equal cost, or equal naturalness in each domain. Consensus formalises restriction (`ConsensusRestriction.v`) and refinement (`ConsensusAdmissibility.v`). GPU memory formalises restriction (`GPURestriction.v`) and refinement (`GPUAdmissibility.v`). The physical-systems case formalises both robust restriction and refinement.

The GPU restriction result should not be read as a full mechanisation of a production GPU memory model or as an independent proof of a DRF-SC theorem. It proves the admissibility repair obtained once the execution class is restricted to data-race-free executions and the DRF-SC bridge is supplied for the toy model. This is the intended level of the case study.

The RCC axioms and application-level coherence constraints are explicit mathematical assumptions. They are not derived from a minimal connection theory. The paper gives a structural criterion, not an automated refinement algorithm and not a decision procedure for admissibility.

Finally, warrant debt is not proposed as a quantitative risk metric here. The development gives warrant debt a qualitative profile: debt kind, repair pressure, and repair cost band. This is enough to distinguish trace debt from boundary debt, or threshold debt from runtime debt. It is not enough to compute expected harm, probability of failure, or economic cost. Those quantitative questions remain future work.

9 Discussion

9.1 Observation interfaces as verification boundaries

A safety property is a property of the relation between the system, the predicate, and the observation interface. If the observation cannot distinguish two states, no claim that separates those states is justified through that observation.

9.2 Refinement as factorisation repair

Refinement is often described as the removal of spurious behaviour from an abstraction. Here it repairs a failed factorisation: it changes the observation boundary so that a predicate invisible at the coarse interface becomes visible at the refined one.

This way of speaking also clarifies why refinement is not always the right repair. Sometimes the better engineering decision is to restrict the claim. Operating envelopes, robust margins, lockout conditions, and restricted modes are not second-class repairs. They are ordinary safety devices. What

matters is that the restricted claim actually factors through the observation being used.

9.3 Warrant debt

Warrant debt is the operational residue of claiming more than the observation can support. The formal object is the collapse witness: same observation, different predicate value. The term is useful only if it prevents equivocation. If the missing distinction is irrelevant to the predicate, there is no debt. If the predicate requires the missing distinction, then calling the discrepancy noise is unsafe.

The development treats warrant debt as a qualitative classification, not merely as a name. A debt profile records the kind of collapsed distinction, the repair pressure, and the expected repair cost band. Quorum debt, trace debt, boundary debt, threshold debt, and runtime debt all instantiate the same factorisation failure, but they do not impose the same engineering burden.

This classification also clarifies what it would mean to carry warrant debt. A project may decide that full observational refinement is too costly. That does not discharge the debt. It changes the safety case. The stronger claim must be rejected or restricted, and the remaining debt must be recorded as a limitation of the supported claim. In this sense, acceptable warrant debt is not hidden debt. It is named, bounded, and attached to a weaker assertion.

The proof obligation remains unchanged: show factorisation, restrict the claim, reject the claim, or refine the observation.

Future work. Future work has three natural directions. The first is quantitative: to refine the qualitative warrant-debt profile into a numerical account of severity, for example by metric distance, information-theoretic loss, or expected repair cost. The second is mechanistic: to replace the DRF-SC bridge in the GPU case with a fuller proof inside a richer memory model. The third is scaling: to move from deliberately small case studies to larger assurance models without changing the factorisation criterion.

10 Conclusion

The paper proves a limited claim. Its consequence is not optional. A safety predicate can be asserted through an observation only if it factors through that observation. If two concrete states have the same observation but different predicate values, the observation does not support the claim. Post-processing cannot repair that loss.

This situation occurs in ordinary safety work. Engineers restrict operating envelopes. They add sensors. They change thresholds. They refine logs.

They reject claims that the evidence cannot support. The paper does not replace those practices. It gives one formal reason why they matter.

The implemented lesson is therefore small but strict. A failed factorisation is not an invitation to tune a post-processor. It is a repair obligation. The obligation may be discharged by restriction, rejection, or refinement. If it is not discharged, it must be carried explicitly as profiled warrant debt.

The warning is simple. An admissibility failure may look like noise. It may look like harmless underspecification. It may look like a modelling convention. But if the safety predicate depends on a distinction the observation has collapsed, the problem is structural. The response is not a stronger post-processor. It is restriction, rejection, or refinement.

A Assumptions

The development contains no admitted lemmas, 41 compiled Rocq source files, and eight named axioms. Every axiom is documented in `ASSUMPTIONS.md` in the accompanying repository. The audit commands that verify this status are:

```
find . -name '*.v' | wc -l    # 41
find . -name '*.vo' | wc -l  # 41
grep -RniE '\bAdmitted\b' . --include='*.v'    # no output
grep -RnE   '\bAxiom\b'     . --include='*.v'   # 8 matches
```

Both grep commands above also return no output when run against `GPURestriction.v` directly, confirming that the restriction repair introduces neither Axiom declarations nor Admitted proofs.

B Rocq File Map

Component	Files	Role
Foundation	Foundation/*.v	Observation, factorisation, mereotopology
Admissibility	Admissibility/*.v	Core admissibility theory
Runtime	Runtime/*.v	Monitor states, fatigue, certificates
Automation	Automation/*.v	Custom proof tactics
Consensus	CaseStudies/ Consensus/*.v	Quorum-safety admissibility, local-quorum restriction
GPU	CaseStudies/GPU/*.v	Thread-local inadmissibility, global refinement, DRF restriction
Physical	CaseStudies/ PhysicalSystems/*.v	Sensor inadmissibility, restriction, refinement
Extraction	Extraction/*.v	Verified OCaml extraction bridge

Table 7: Rocq source file map.

References

- [1] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2), 2014.
- [2] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4):14:1–14:64, 2011.
- [3] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification*, LNCS 1855, pages 154–169. Springer, 2000.
- [4] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement for symbolic model checking. *Journal of the ACM*, 50(5):752–794, 2003.
- [5] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM Symposium on Principles of Programming Languages*, pages 238–252, 1977.
- [6] International Electrotechnical Commission. *IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems*. IEC, current multi-part standard.
- [7] International Organization for Standardization. *ISO 31000:2018 Risk management, Guidelines*. ISO, 2018.

- [8] Leslie Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):51–58, 2001.
- [9] Scott Owens, Susmit Sarkar, and Peter Sewell. A better x86 memory model: x86-TSO. In *Theorem Proving in Higher Order Logics*, LNCS 5674, pages 391–407. Springer, 2009.
- [10] David A. Randell, Zhan Cui, and Anthony G. Cohn. A spatial logic based on regions and connection. In *Proceedings of the 3rd International Conference on Principles of Knowledge Representation and Reasoning*, pages 165–176, 1992.
- [11] The Coq Development Team. *The Coq Proof Assistant*. Zenodo, 2024.